



面向AI推理负载的异构存储管理 系统优化

汇报队伍：古法编程继承人
成员：井光成，徐子淳，祁馨叶
指导老师：王亚纲，王小银



目录

1.

项目背景与赛题目标

2.

系统架构设计

3.

实现亮点与难点

4.

测评与结果

5.

分工与时间线

6.

ai使用情况说明



目录

1.

项目背景与赛题目标

2.

系统架构设计

3.

实现亮点与难点

4.

测评与结果

5.

分工与时间线

6.

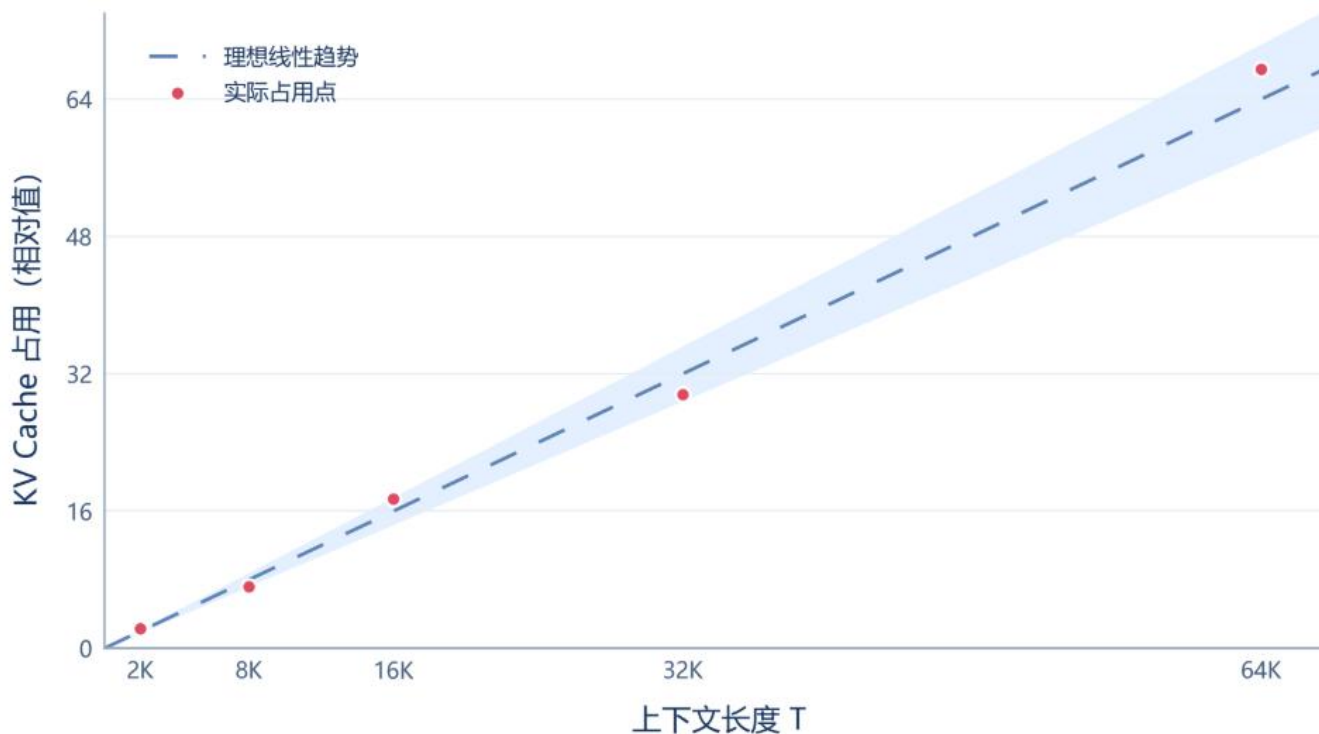
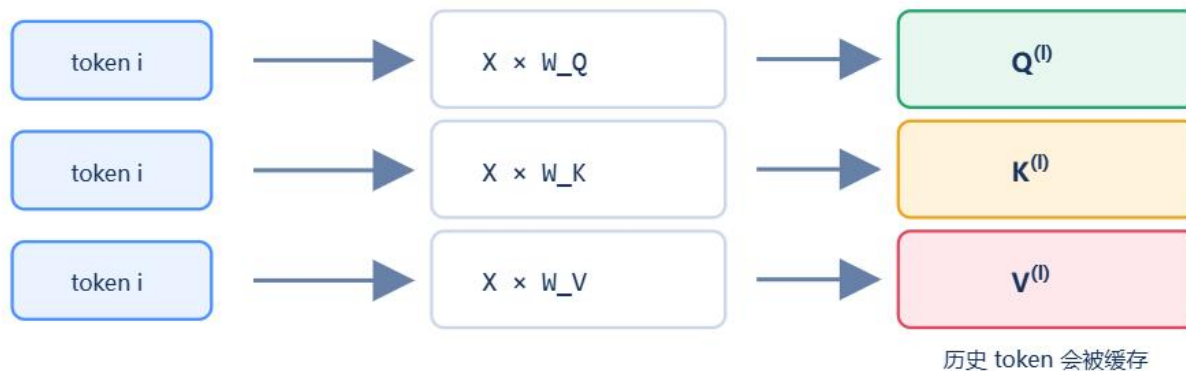
ai使用情况说明

项目背景

问题来源

- LLM 推理需要保存历史 token 的 Key-Value 缓存，以减少重复 Attention 计算。
- 当上下文从 2K、8K 增长到 16K、32K 甚至更高时，KV Cache 会近似线性膨胀。
- 在资源受限设备上，KV Cache 会逐渐挤占模型权重、激活与运行时调度所需要的显存预算。

第 l 层输入: $X^{(l)}$ ($l = 1, \dots, L$) 分别乘以三个固定的权重矩阵



赛题任务

任务一

监控访问模式，定义块级热度

观察访问频率、最近访问时间等信号，为 KV Cache 的冷热分层提供基础依据。

任务二

热留显存，温迁内存，冷迁外存

根据访问行为在 GPU、CPU、SSD 之间组织动态迁移与命中后回填。

任务三

设计块级存储分配与碎片控制

以 block 为单位组织存储，减少动态增长带来的显存和内存碎片问题。

目标

尽量少改框架，提升吞吐并降低等待

在尽量透明集成的前提下，支持更长上下文，并减少 GPU 等待迁移的时间。



目录

1.

项目背景与赛题目标

2.

系统架构设计

3.

实现亮点与难点

4.

测评与结果

5.

分工与时间线

6.

ai使用情况说明

总体架构设计

设计思路

MiniFlex 采用“接入层 - 任务层 - 缓存 / 存储 / 传输层”的分层架构，通过 vLLM V1 Connector 接入推理流程，在尽量少侵入原框架的前提下，统一管理 KV Cache 的命中、迁移、回填与复用。

关键机制

- 任务化管理

将 GET、PUT、回填和完成回调抽象为 KVTask，统一维护任务状态与资源释放。

- 块级缓存管理

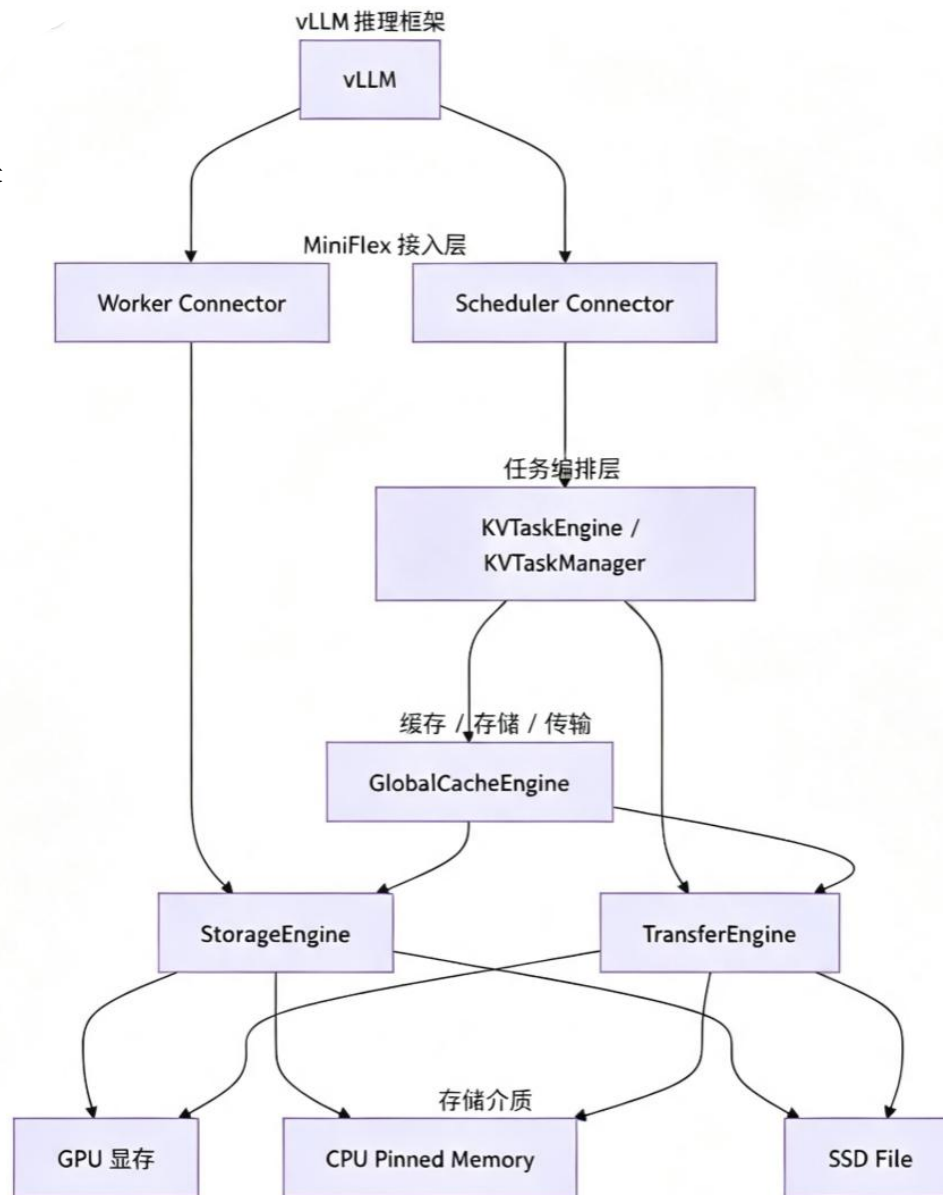
以 block 为基本单位维护前缀匹配、分配、淘汰与回收，通过 ready / pending 状态避免临时 miss。

- 多级存储协同

统一管理 GPU 显存、CPU Pinned Memory 与 SSD File，支持 KV 在不同介质之间迁移和回填。

- 布局自适应

根据 vLLM GPU KV Cache 的实际布局组织传输，兼容 LAYERBLOCK 等版本变化。





目录

1.

项目背景与赛题目标

2.

系统架构设计

3.

实现亮点与难点

4.

测评与结果

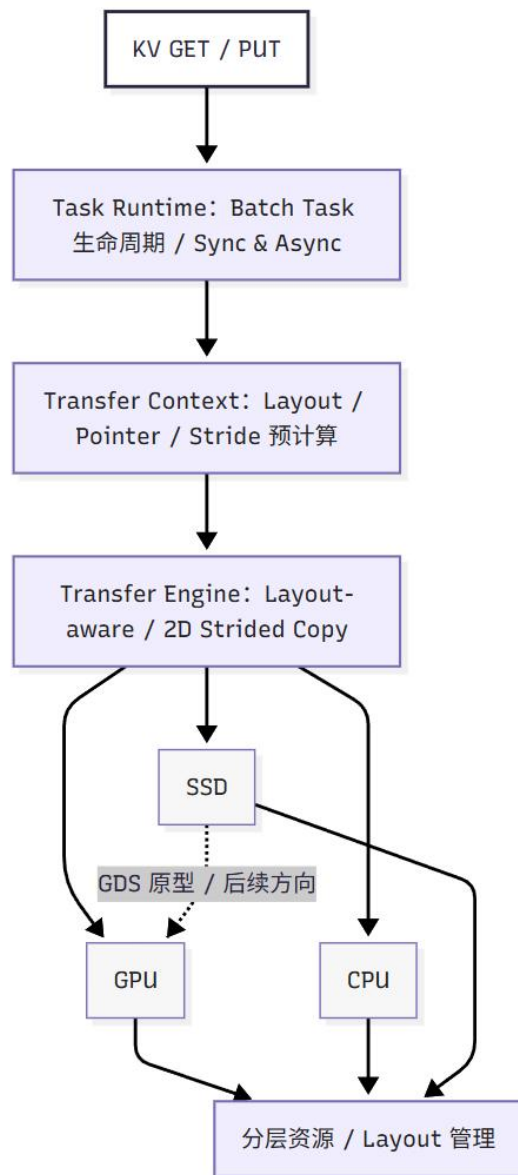
5.

分工与时间线

6.

ai使用情况说明

KV Cache 分层传输链路优化

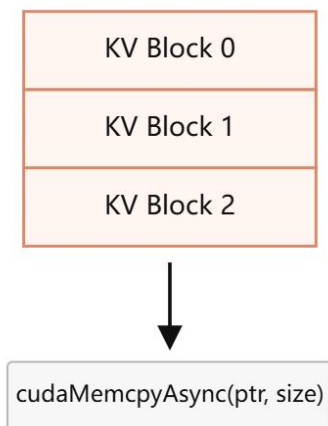


- 围绕 **Layout**、**Transfer**、**Task Runtime** 和 **Storage** 四个层次，对 KV Cache 搬运链路进行系统性重构

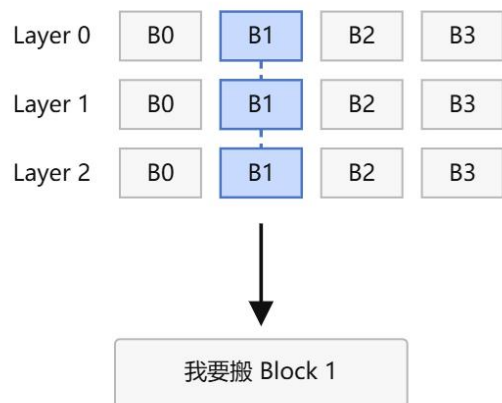
创新设计(1/4): KV Layout 解耦

Problem: 固定连续假设失效

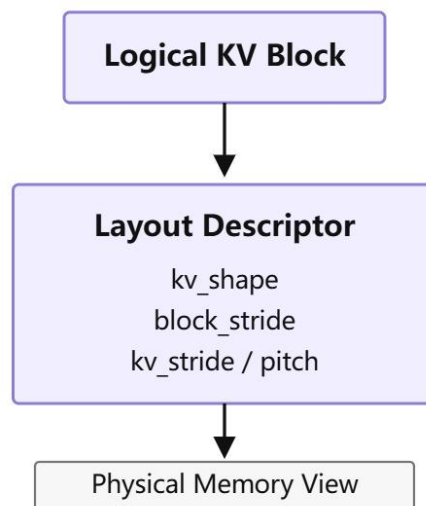
旧布局: LAYERFIRST / 连续假设



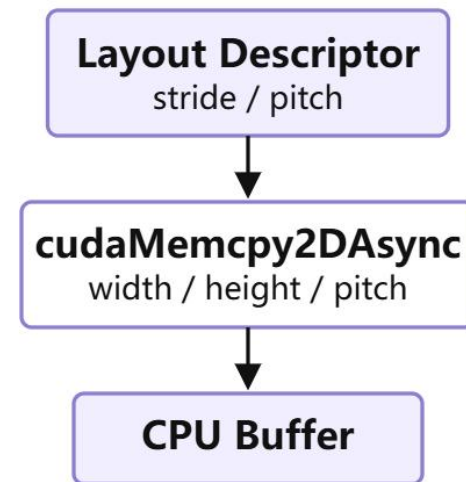
新版布局: LAYERBLOCK



Core: Logical Block → Physical Layout



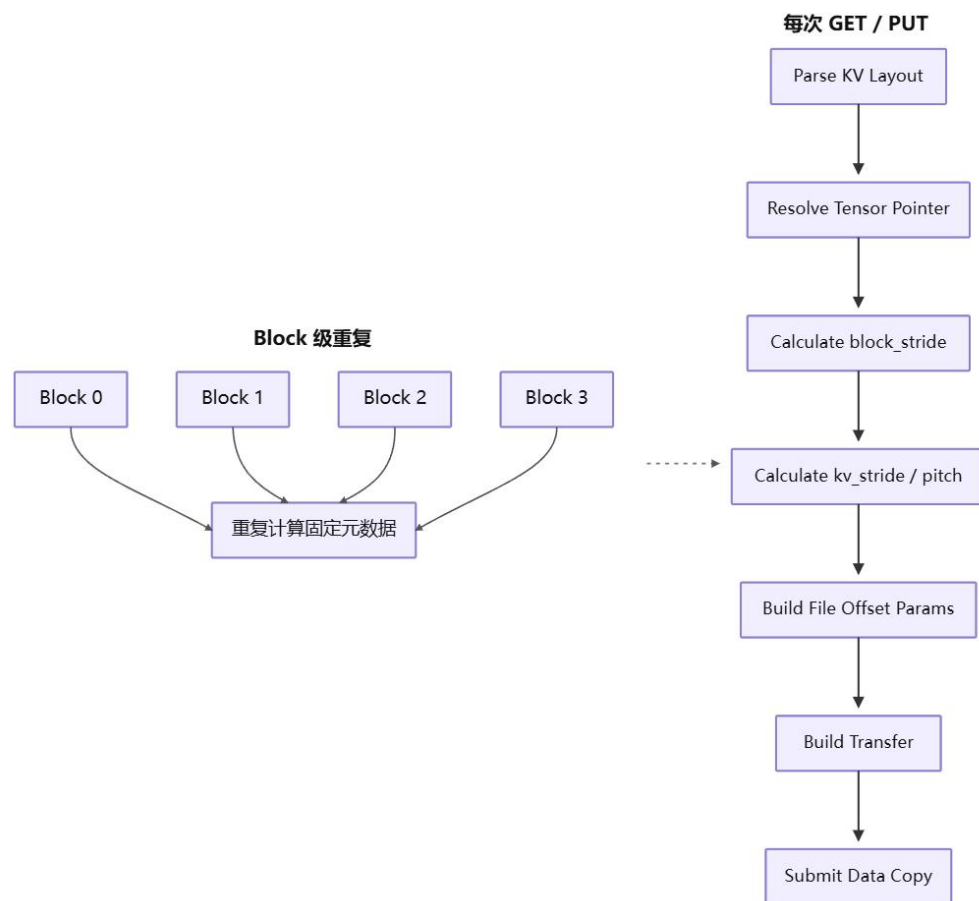
Solution: 2D Strided Copy



底层 Transport 不再依赖固定 Layout 假设, 而是依据 stride / pitch 完成真实物理布局搬运。

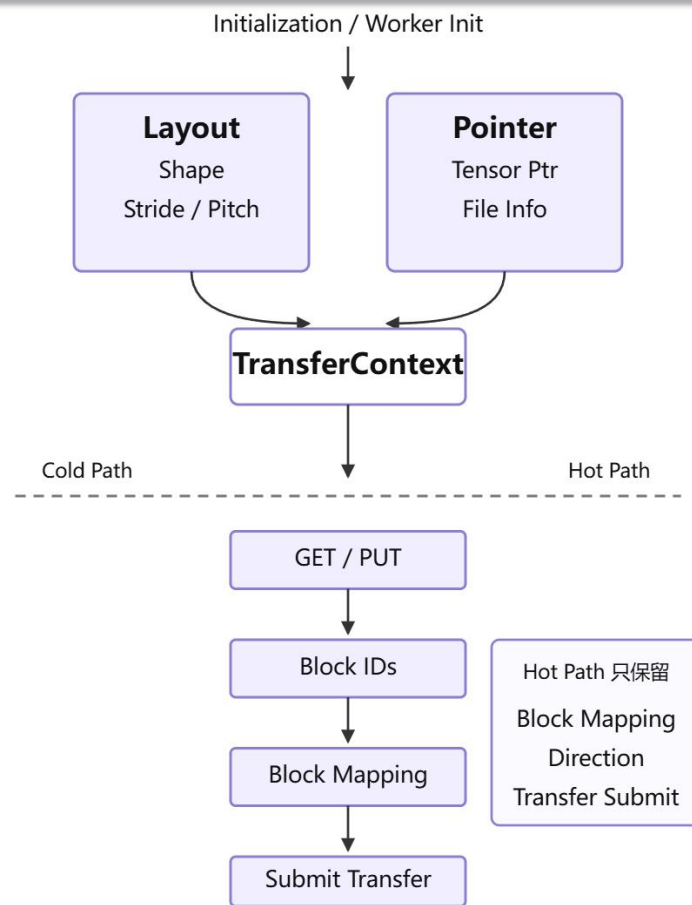
创新设计(2/4): 传输热路径优化

Before: 请求时重复解析



高频 block 搬运中, 固定元数据被反复计算, 热路径偏重。

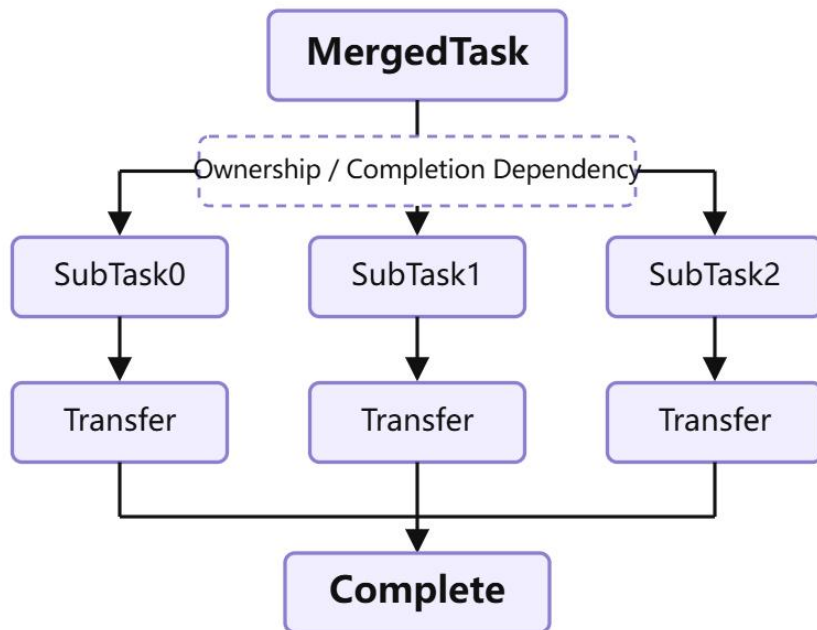
After: 固定元数据初始化前移



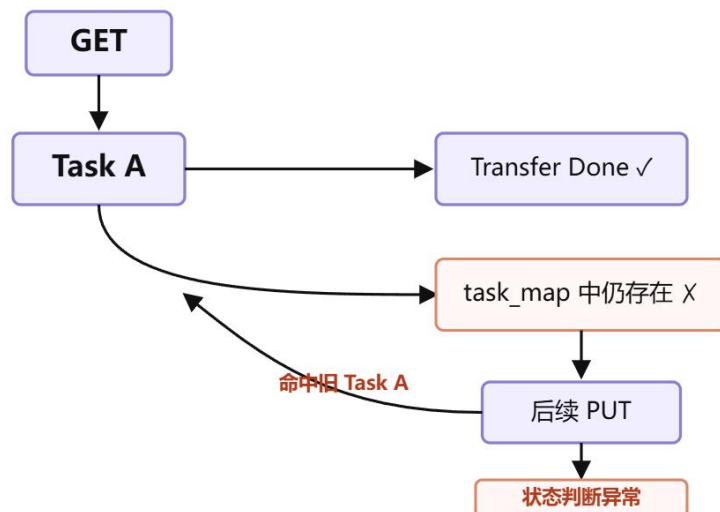
将 Layout、Pointer、Stride / Pitch 等固定元数据前移到初始化阶段, 使 GET / PUT 热路径更接近纯数据搬运。

创新设计(3/4): Batch Task 生命周期

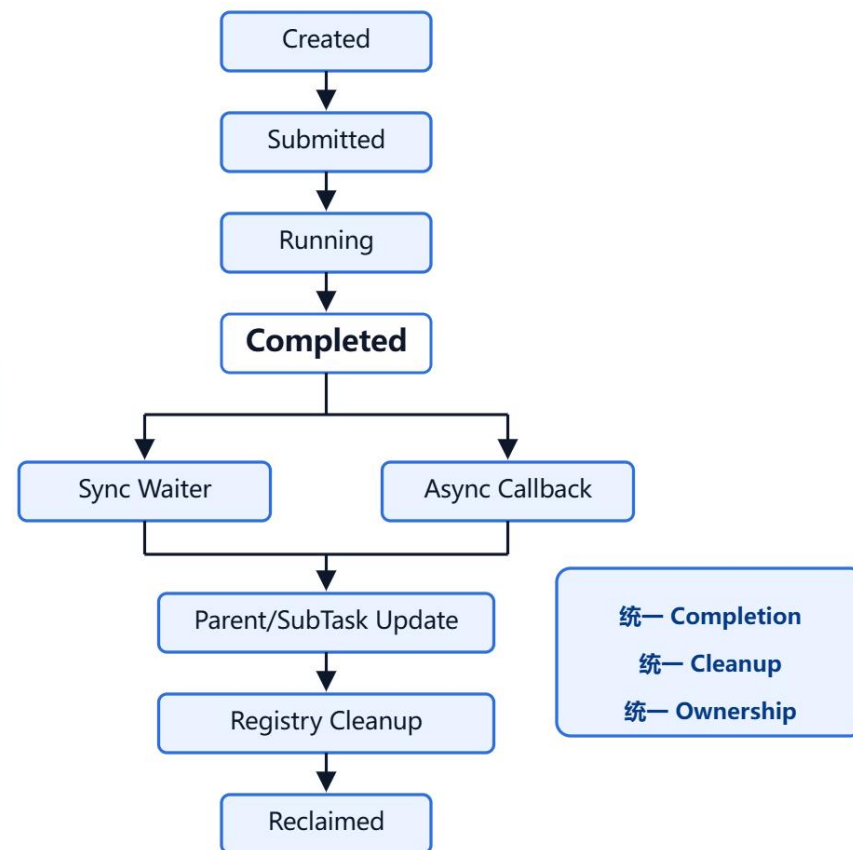
Batch Task Tree



Data Completed $\neq$ Task Reclaimed



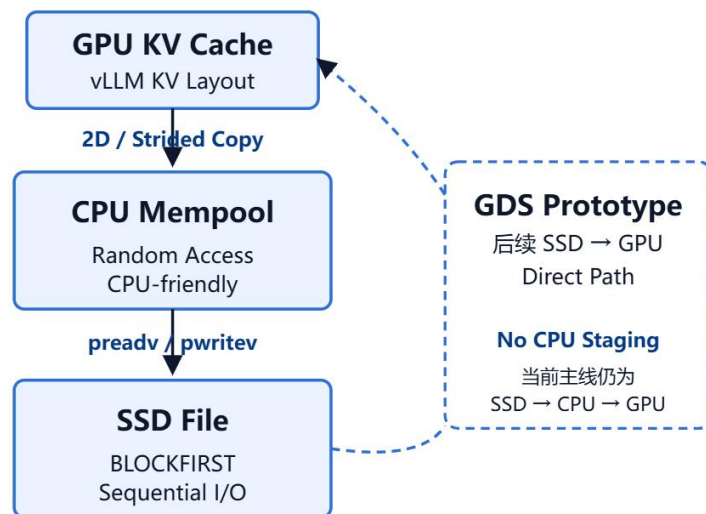
Unified Lifecycle



Batch 后复杂的不是任务合并本身，而是 merged/sub-task 在同步与异步路径下保持一致的完成、归属和清理语义。

创新设计(4/4): Layout 与 Resource 解耦

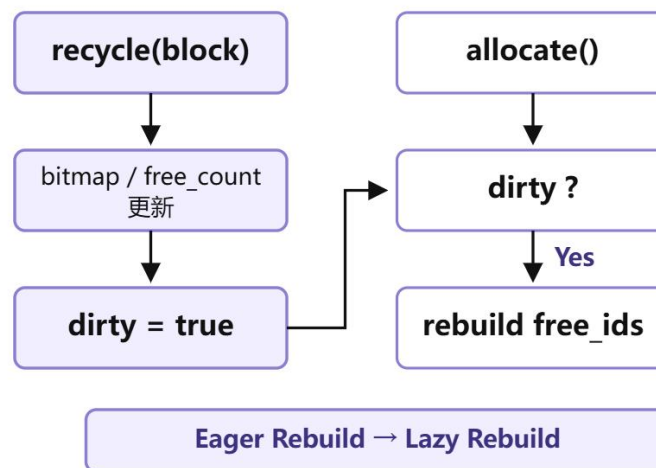
GPU / CPU / SSD 分层存储路径



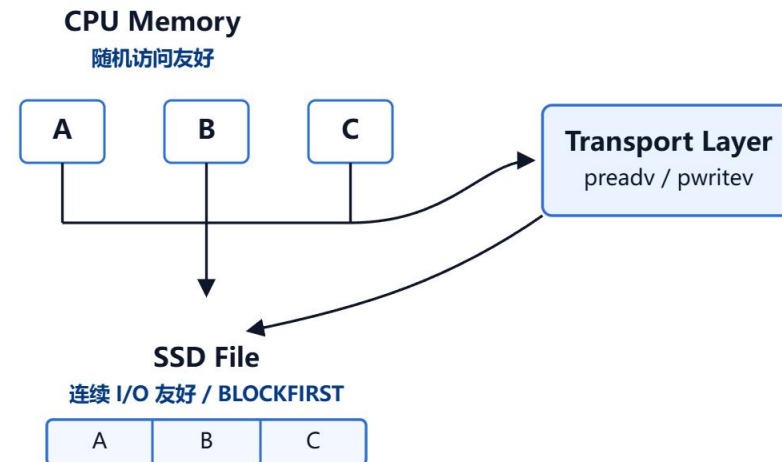
分层资源 / Layout 管理

不同存储层按各自访问特点组织数据, 由传输层完成衔接

Mempool: Eager Rebuild → Lazy Rebuild



CPU / SSD 异构 Layout Bridging

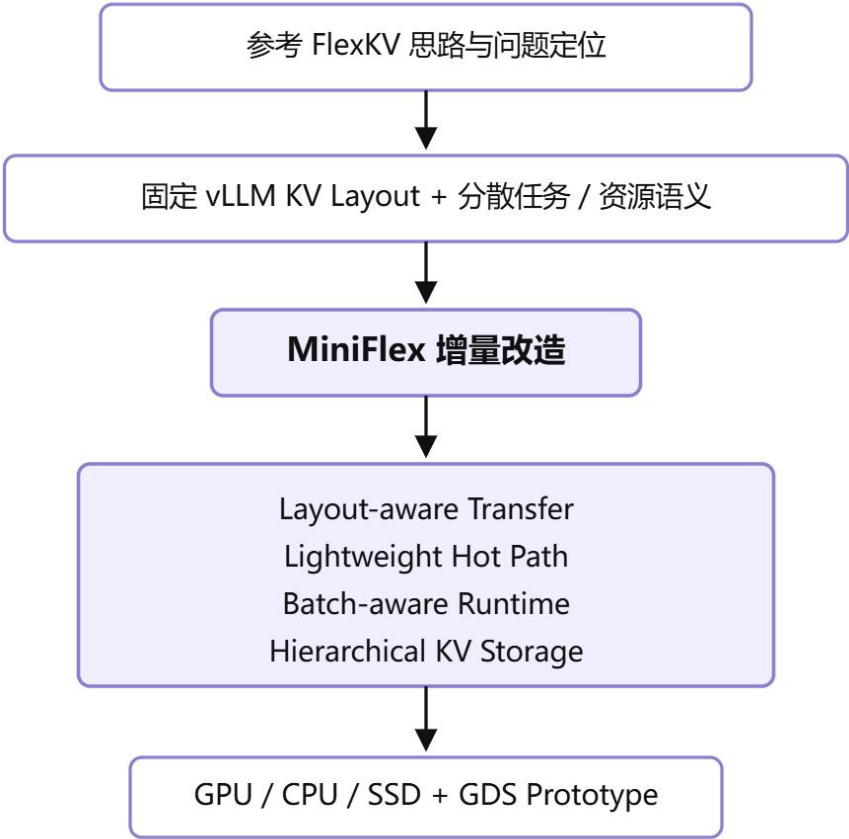


不强求存储层 Layout 同构, 由 Transport Layer 衔接 CPU 随机访问与 SSD 连续 I/O

从特定布局适配到分层 KV Transfer Runtime

主线的改造前后

维度	Before	After
KV Layout	固定布局假设	Stride / Pitch Layout Descriptor
Transfer	请求时重复解析	TransferContext固定元数据预计算
Task	Sync / Async 分散处理	统一 Batch Lifecycle
Storage	CPU / SSD Layout 耦合	异构 Layout + Path-aware Resource



KV Cache 系统从“针对某种布局搬数据”，演进为“描述数据布局、选择传输路径、管理任务与资源”的通用分层传输 Runtime。

难点一：GPU↔CPU 传输策略选择

实现创新点1、2时

◆ 问题

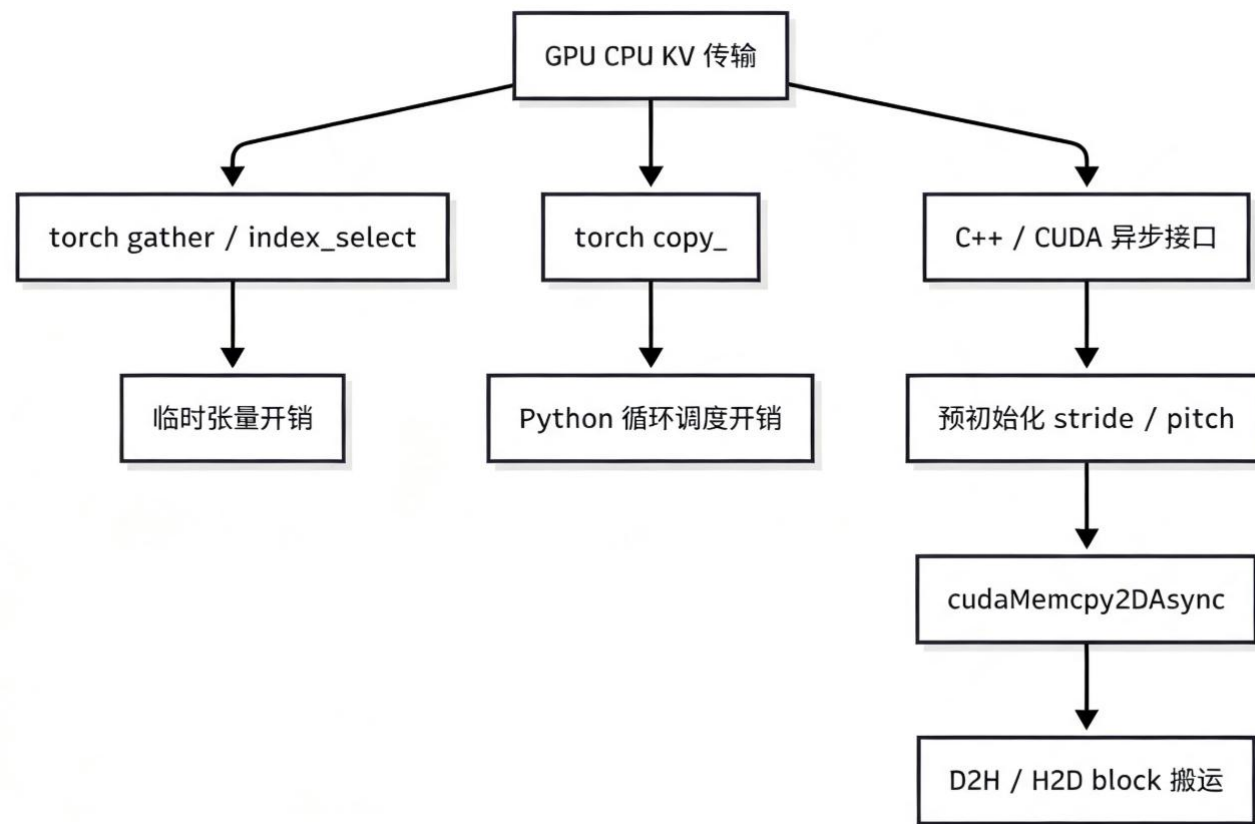
KV block 在 GPU 与 CPU 中按不同 layout 组织，传输时既要保证 block 对齐和数据正确，又要尽量减少 Python 层调度和临时张量开销。

◆ 方案对比

- ◆ • torch gather / index_select
实现简单，但会产生临时张量，且跨布局搬运开销较高。
- torch copy_
可逐 block 拷贝，但 Python 循环调度成本明显。
- C++ / CUDA 异步接口
预初始化 tensor 指针、stride、pitch 等布局信息
使用 cudaMemcpy2DAsync 完成 D2H / H2D 搬运。

◆ 最终选择

GPUCPUTransferCTX + cudaMemcpy2DAsync
将布局适配和块级传输下沉到 C++ / CUDA，减少 Python 热路径开销。



难点二：GDS资源规划

实现创新点4时

◆ 问题

当前 SSD 命中路径为 DISK2H -> H2D, 需要 CPU staging block。

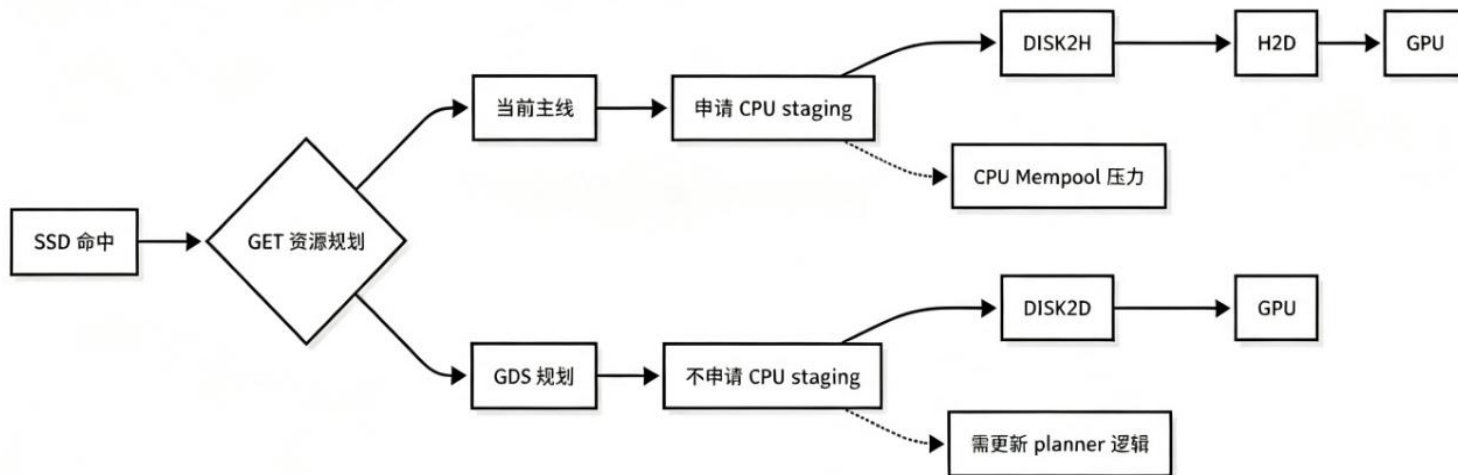
如果引入 GDS, SSD 可以通过 DISK2D 直达 GPU, 理论上不再需要 CPU staging。

◆ 资源规划矛盾

- ◆ 沿用原 GET 规划：仍会申请 CPU block, 削弱 GDS 直连收益
- ◆ 改为 DISK2D：需要重新规划 src / dst block、传输类型和回调
- ◆ CPU 空间不足时，不应影响 SSD -> GPU 直达路径

◆ 当前处理

GDS 相关 DISK2D / D2DISK 和 GDSTransferWorker 已作为原型保留；当前主线 GET 仍采用 DISK2H -> H2D, GDS 作为后续接入方向。



难点三：GDS后续接入中的CPUCache晋升取舍

实现创新点4时

◆ 初始设想

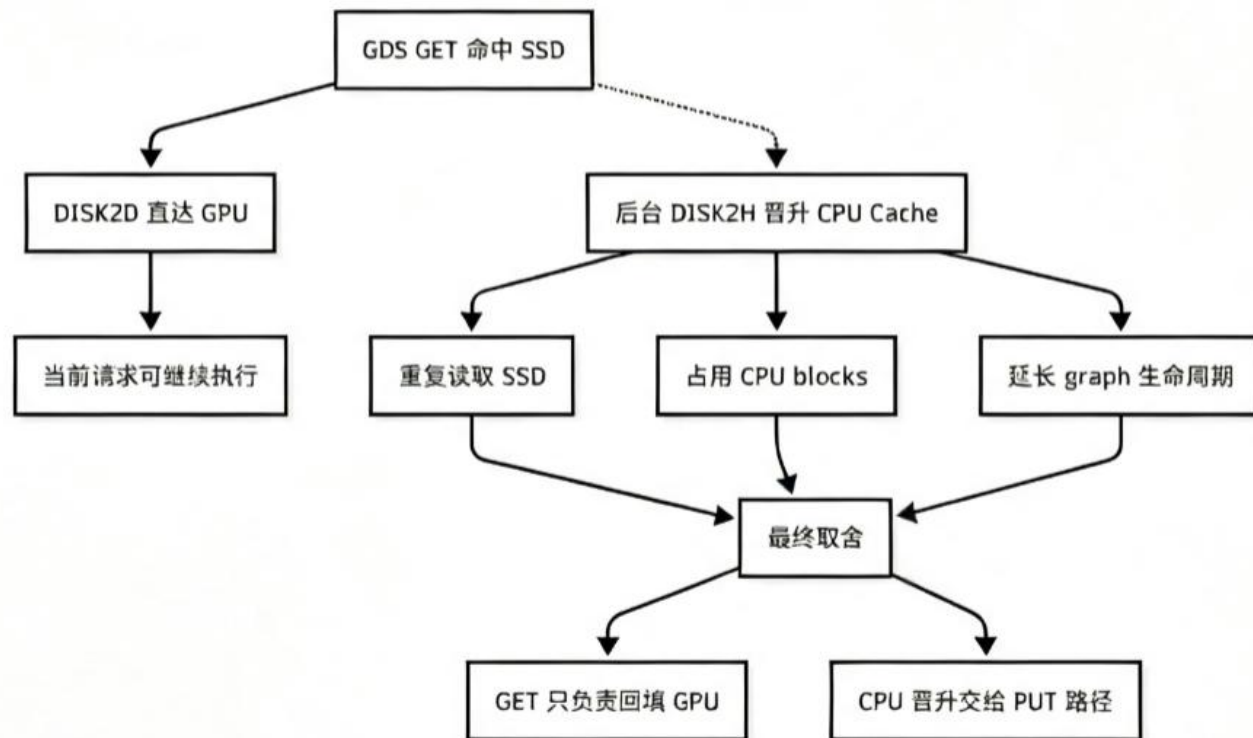
DISK2D 负责当前请求的 SSD -> GPU 直达回填；
同时后台 DISK2H 将同一份 KV 晋升到 CPU Cache，避免后续请求再次访问 SSD。

◆ 问题

- ◆ 同一份 KV 需要重复读取 SSD
- 后台 DISK2H 会拉长 lock、buffer 和 graph 生命周期
- 需要额外 CPU block，可能受到 CPU Mempool 限制
- task-end 与 graph-complete 分离，资源释放更复杂

◆ 最终取舍

GET 阶段只服务当前请求，负责 SSD -> GPU 直达回填；
CPU Cache 晋升交给请求结束后的 PUT 路径处理。





目录

1.

项目背景与赛题目标

2.

系统架构设计

3.

实现亮点与难点

4.

测评与结果

5.

分工与时间线

6.

ai使用情况说明

测评方法与 Benchmark

- ◆ 本项目测评围绕命中、迁移、回填、超容量稳定性展开采用场景化 benchmark 与场景化 baseline, 针对不同系统行为选择对应的对比对象
- ◆ 采用自定义可控 benchmark, 通过 vLLM OpenAI 接口发起请求, 统计 TTFT、miss、CPU / SSD 命中和回填行为。
- ◆ 核心指标: TTFT、median / p90、miss 比例、CPU hit blocks、SSD hit blocks、H2DISK blocks
- ◆ 自定义负载用于精确控制 prompt 长度、共享前缀、工作集规模和缓存容量, 从而稳定观察 MiniFlex 的分层缓存行为。

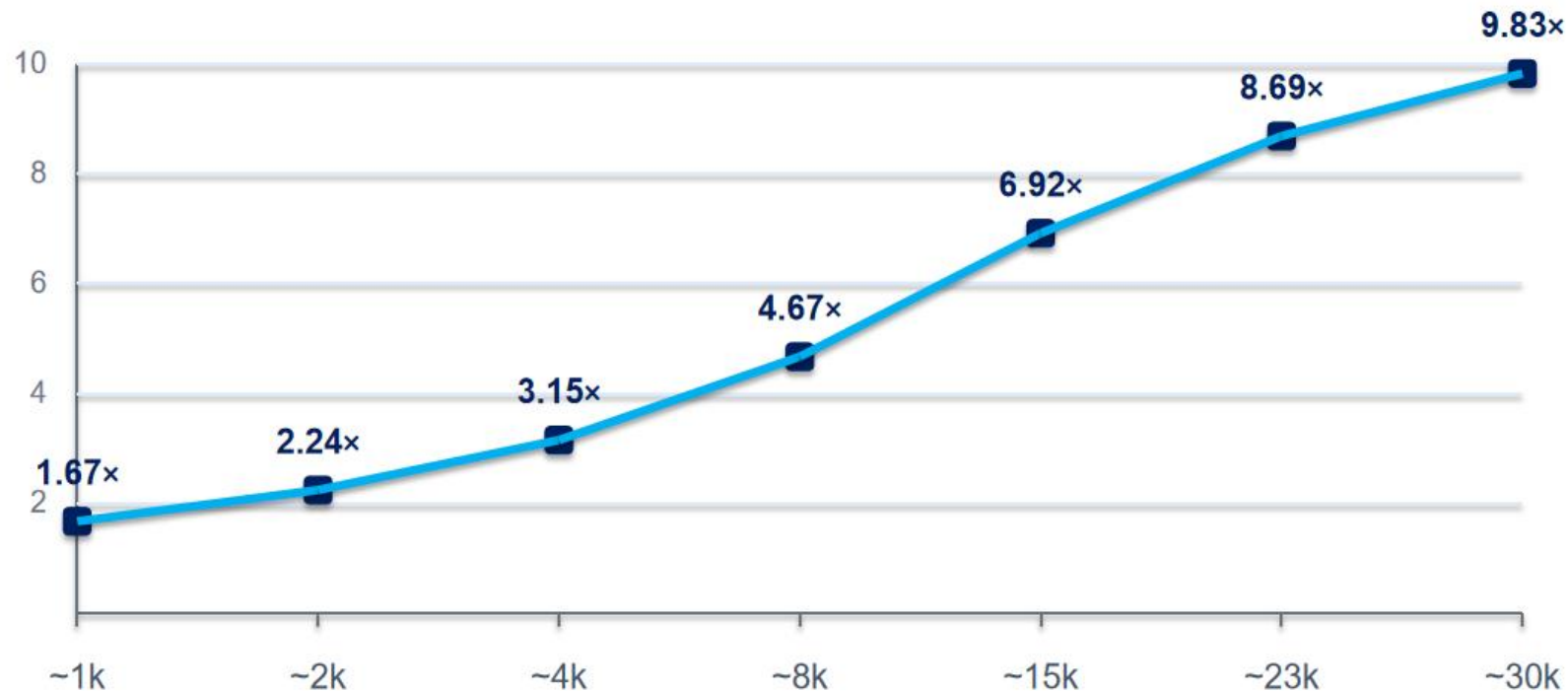
Benchmark	测试目标	Baseline
bench_ttft.py	长上下文命中收益	冷态重算
bench_overflow.py	超 GPU KV 容量稳定性	vLLM APC
bench_mixed.py	热点 + 长尾混合负载	APC / MiniFlex / APC + MiniFlex
bench_ssd_e2e.py	SSD 持久化与回填闭环	冷重算 / SSD恢复 / CPU回填命中

RTX 5090、Qwen3-8B 和 vLLM 0.23

长上下文加速效果

在长上下文场景中，MiniFlex 命中后可以显著减少全量重算开销，而且上下文越长收益越大

加速比（热态命中 vs 冷态重算）



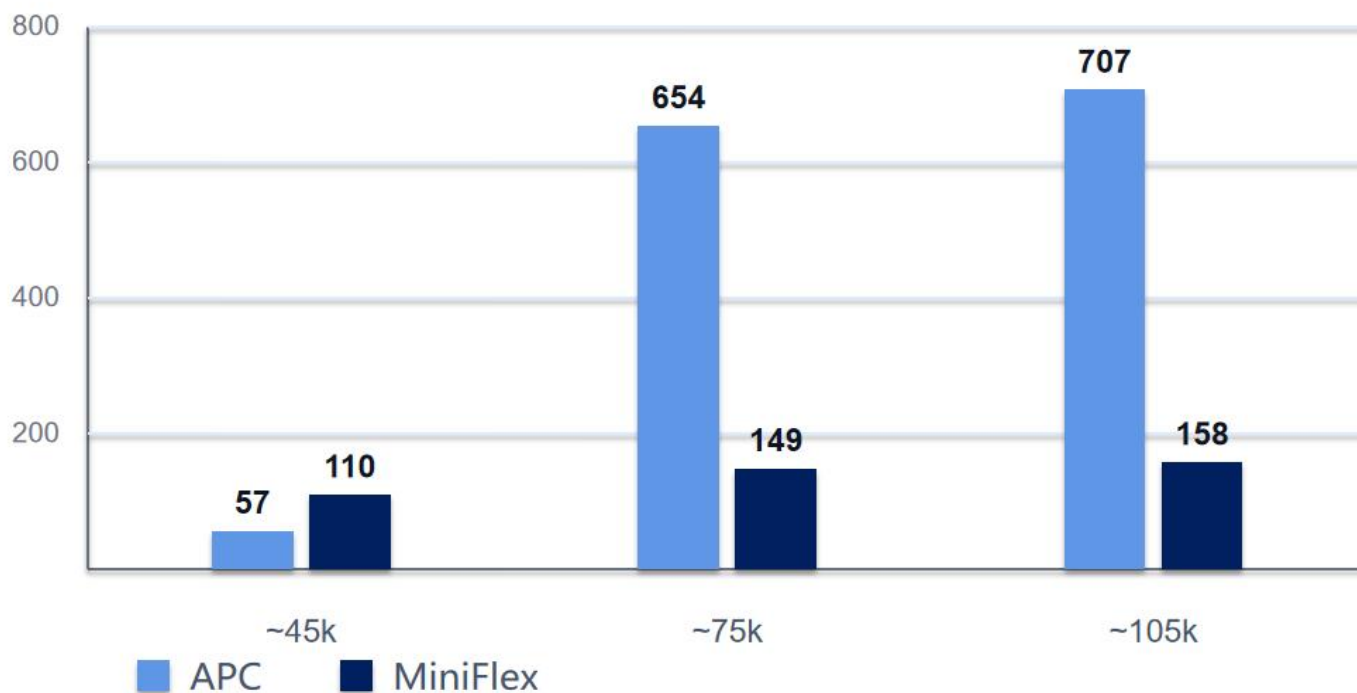
结论

- 加速收益随上下文长度持续扩大
- 省下的重算比例从 40% 提升到约 89.8%
- 约 30k 上下文场景下, TTFT 从 3858.0 ms 降到 392.4 ms

容量优势与 APC 对比

一旦工作集超出 GPU 容量, MiniFlex 的 CPU-backed 复用路径会明显比原生 APC 更稳定

TTFT 对比 (ms)



- ~45k 工作集仍装得进GPU 时, APC 命中更快
- 一旦工作集超过 GPU KV 容量, APC 迅速出现 100% miss
- MiniFlex 通过 CPU 层承接溢出, 仍稳定在 149-158 ms 且 0% miss

混合负载与指标体系

在热点与长尾共存的真实负载里，MiniFlex 与 APC 组合能在速度与容量之间取得更平衡的效果

混合负载结果（TTFT, ms）

配置	hot	tail	overall
APC	60	707	515
MiniFlex	103	177	153
APC + MiniFlex	67	188	165

结果

- APC 更擅长热点数据留在 GPU 的极快命中
- MiniFlex 更擅长长尾超容量场景的稳定复用
- 两者叠加后更贴近真实业务的混合负载

当前指标体系

- 服务可用性：成功率、失败率
- 核心性能：TTFT、TPOT、吞吐
- 缓存行为：KV 使用率、prefix 命中
- 资源压力：GPU / CPU 占用与上下文分档

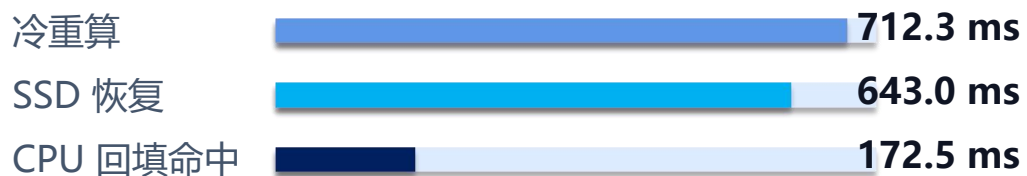
SSD 持久化与 CPU 回填验证

CPU Cache 淘汰后从 SSD 恢复、再回填 CPU 的完整多级缓存闭环

SSD E2E 验证流程



TTFT p50 对比



结论

- SSD 层能保存超过 CPU 容量的完整工作集
- 恢复后自动回填 CPU，后续访问直接走更快路径
- greedy 输出token ID 一致，验证回填内容正确性

开源成果展示

Validate block id range in Mempool.recycle_blocks #166

Merged linhu-nv merged 2 commits into taco-project:main from xianweihuihuan:fix/mempool-recycle-range-check on May

Conversation 3 Commits 2 Checks 1 Files changed 1

fix: handle shutdown signal in inner batch loop of Tran #172

Merged linhu-nv merged 1 commit into taco-project:main from xianweihuihuan:fix/transfer-worker-shutdown-

fix: move critical section updates inside lock in SharedOp #173

Merged linhu-nv merged 1 commit into taco-project:main from xianweihuihuan:fix/lock-critical-section

Conversation 1 Commits 1 Checks 2 Files changed 1

fix: update task.slot_mapping in _set_slot_mapping_imp

Merged linhu-nv merged 1 commit into taco-project:main from xianweihuihuan:fix/kvtask-slot-mapping-not-upd

Conversation 2 Commits 1 Checks 2 Files changed 1

fix: cancel KVTask immediately when match/put cond prevent memory leak #176

Merged linhu-nv merged 2 commits into taco-project:main from xianweihuihuan:fix/cancel-kvtask-on-zero-

Conversation 2 Commits 2 Checks 4 Files changed 1

refactor: own sub-tasks at batch merge to fix batch task lifecycle #184

Merged linhu-nv merged 7 commits into taco-project:main from xianweihuihuan:refactor/batch-task-lifecycle 4 days ago

Conversation 1 Commits 7 Checks 4 Files changed 6

fix(cache): avoid unnecessary CPU block allocation for GDS GET #229

Merged linhu-nv merged 1 commit into taco-project:main from xianweihuihuan:fix/gds-get-cpu-allocation last week

Conversation 0 Commits 1 Checks 2 Files changed 2

fix: support vLLM 0.23+ non-MLA KV layout via new LAYERBLK #187

Merged linhu-nv merged 2 commits into taco-project:main from xianweihuihuan:fix/vlm023-nonmla-layout on Jun 30

Conversation 0 Commits 2 Checks 4 Files changed 2

fix(kvtask): release tasks after early response and graph completion #230

Merged linhu-nv merged 2 commits into taco-project:main from xianweihuihuan:fix/request-returned-lifecycle last week

Conversation 0 Commits 2 Checks 2 Files changed 2

xianweihuihuan commented 3 weeks ago

xianweihuihuan commented last week

xianweihuihuan commented 2 weeks ago

本项目参考了开源项目FlexKV的部分思路,但我们并非复现代码,而是学习思路重写,且做了多面向实际推理场景的增量改造。在项目开发期间,我队积极与FlexKv开发者沟通,持续参与FlexKV相关问题定位、修复与验证,并累计形成 **10 项公开开发协作记录**,其中包括 **9 项**已合并 PR、**1 项**已修复 Issue,体现了我队对项目的持续投入。



目录

1.

项目背景与赛题目标

2.

系统架构设计

3.

实现亮点与难点

4.

测评与结果

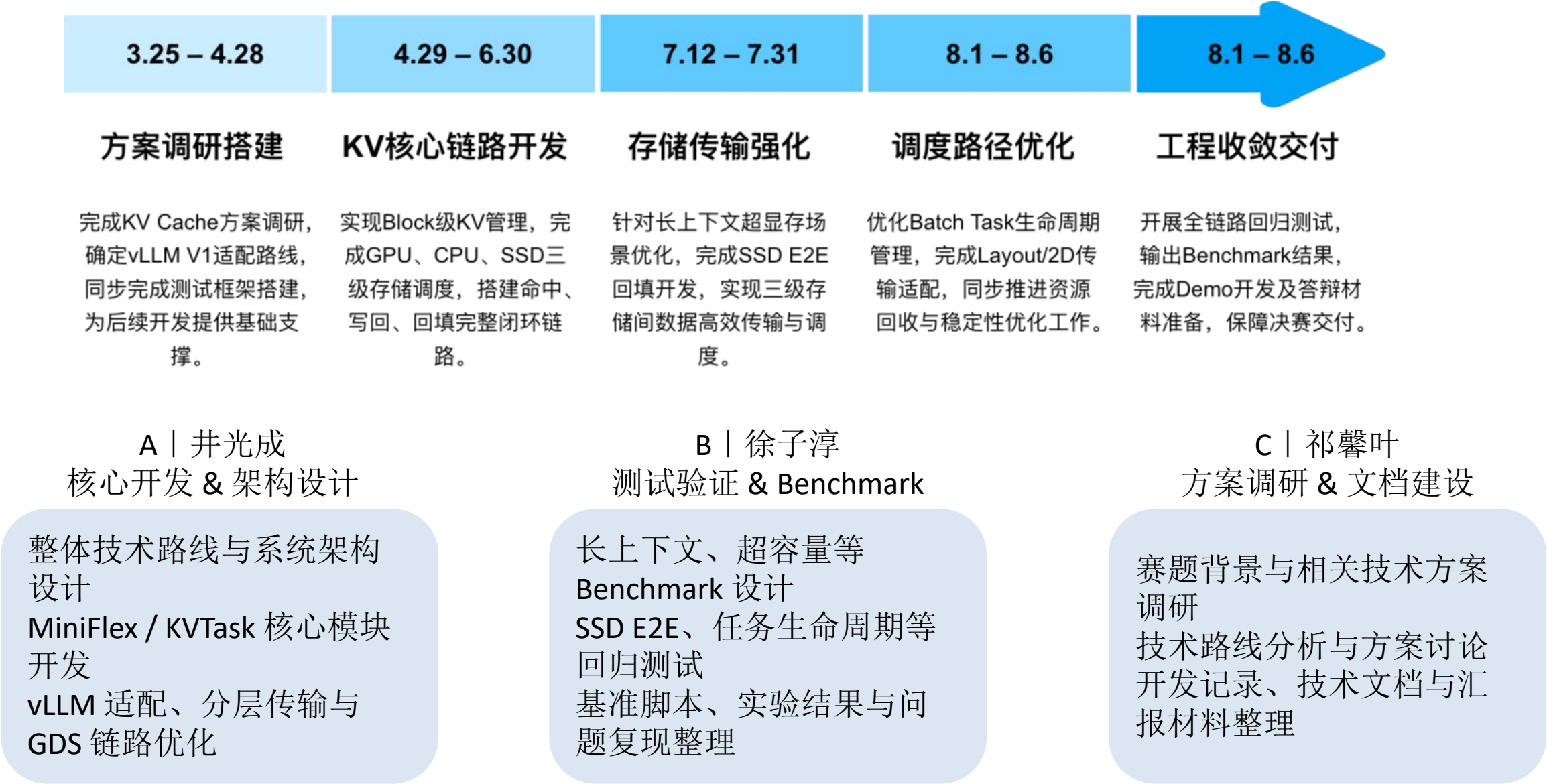
5.

分工与时间线

6.

ai使用情况说明

项目时间线与分工





目录

1.

项目背景与赛题目标

2.

系统架构设计

3.

实现亮点与难点

4.

测评与结果

5.

分工与时间线

6.

ai使用情况说明

AI 工具使用说明与披露

本队将 AI 定位为辅助工具

◆ 当前使用场景

- 在写项目测试脚本时，我队使用了 AI 助手辅助，告诉ai思路后，让ai编写，人工复核
- 使用模型：claude, codex

◆ 人工复核原则

- AI 输出内容由本队成员结合代码实现和实验结果进行人工核对
- 所有技术表述、测试结论和答辩内容均以人工复核结果为准

◆ 记录留存

- 当前已额外整理 AI 使用文件，见 ``miniflex/docs/test_overview.md``



恳请各位老师批评指正